```
    spec:
      containers:
      - name: gpt-4
        image: <beharaLLM-registry>/<beharaLLM-image>
        ports:
        - containerPort: 5000
```

```[_{{{CITATION{{{_5{Deploying Large Language Models on Kubernetes: A ... - behara.AI](https://www.behara.ai/deploying-large-language-models-on-kubernetes/)

Next, create a Kubernetes service to expose the deployment:

```
Yaml

apiVersion: v1

kind: Service
metadata:
  name: gpt-4-service
spec:
  selector:
    app: gpt-4

  ports:
  - protocol: TCP
    port: 80
    targetPort: 5000
  type: LoadBalancer
```

```[_{{{CITATION{{{_6{Deploying Large Language Models on Kubernetes: A ... - behara.AI] (https://www.behara.ai/deploying-large-language-models-on-kubernetes/)

Apply the deployment and service YAML files to the Kubernetes cluster as follows:

```
Yaml

kubectl apply -f deployment.yaml

kubectl apply -f service.yaml
```

```[_{{{CITATION{{{_7{Deploying Large Language Models on Kubernetes: A ... - behara.AI](https://www.behara.ai/deploying-large-language-models-on-kubernetes/)

## Industry usage of LLM on Kubernetes

Industries are increasingly leveraging LLMs on Kubernetes for various applications. Here are a few examples.

***Healthcare:*** Hospitals and healthcare providers use LLMs for tasks like medical record analysis, patient support, and diagnostic assistance. Kubernetes helps manage the computational demands of these models, ensuring efficient and scalable deployment.

Table 1: A comparison between deploying LLM on Kubernetes and in business applications

| | LLM on Kubernetes | LLM in business applications |
|---|---|---|
| **Scalability** | High scalability with Kubernetes, allowing easy scaling up or down based on demand. | Scalability depends on the infrastructure and resources available within the business environment. |
| **Resource management** | Efficient resource management with Kubernetes, isolating resources for optimal performance. | Resource management may vary, depending on the business's IT infrastructure and capacity |
| **Deployment complexity** | Complex set up and configuration required for Kubernetes. | Deployment can be simpler, depending on the existing infrastructure and tools. |
| **Cost efficiency** | Can be cost-efficient due to optimised resource usage and reduced manual intervention. | Costs can vary based on the scale of deployment and the need for additional infrastructure. |
| **Security** | Enhanced security with Kubernetes' network policies and access control. | Security measures depend on the business's cybersecurity practices and protocols. |
| **Flexibility** | High flexibility with various Kubernetes environments. | Flexibility depends on the business's specific needs and existing systems. |
| **Maintenance** | Requires ongoing maintenance and monitoring. | Maintenance depends on the business's IT support and resources. |
| **Use cases** | Suitable for large-scale applications requiring high availability and resilience. | Suitable for various business applications, including customer support, data analysis, and more. |
```